Übungsblatt 9: Paralleles Rechnen & Multicore

Mikrocomputertechnik 3 · 32 Orientierungspunkte · ca. 75–90 Minuten

Aufgabe 1: MESI mit drei Kernen (6 Punkte)

Drei Write-Back-Caches, anfangs alle I, unterer Speicher L enthält vier. Reads ohne weitere Kopie ergeben E. Fremdread auf M schreibt hier nach L zurück, alle beteiligten Kopien werden S. Vor M-Auswurf wird zurückgeschrieben.

1. Ergänzen Sie Zustände samt Werten und L nach jeder Aktion. (3 P)
2. Welchen unteren Wert darf es nach dem ersten Write geben? Welchen Wert muss der folgende fremde Read liefern? (1 P)
3. Welche fremden Kopien muss Kern 0 bei seinem späteren Write ungültig machen? (1 P)
4. Zwei S-Kopien bleiben übrig, eine wird sauber ausgeworfen. Muss die verbleibende automatisch E werden? (1 P)

Aktion	Kern 0	Kern 1	Kern 2	L
Kern 2 liest				
Kern 2 schreibt 9				
Kern 0 liest				
Kern 1 liest				
Kern 0 schreibt 13				
Kern 0 wirft aus				
Kern 1 liest				

Ihre Lösung:

Aufgabe 2: Acht-Byte-Objekte in einer großen Zeile (4 Punkte)

128-Byte-Cachezeilen, Basis `0x30004000`. Kern 0 schreibt nur a bei Offset acht, Kern 1 nur b bei Offset 24; beide Objekte sind acht Byte groß und getrennt zugeteilt. Anfangs besitzt Kern 1 die Zeile M.

1. Geben Sie Objektadressen und gemeinsamen vollständigen Zeilenbereich an. (1 P)
2. Schreibfolge 1, 0, 0, 1, 0, 1: Wie oft wechselt der Besitzer? (1 P)
3. b wird bei Offset 128 platziert. Neue Adresse? Warum reichte Basisausrichtung allein nicht? (1 P)
4. Repariert das auch verlorene Updates auf einem gemeinsam ungeschützten Zähler? (1 P)

Ihre Lösung:

Aufgabe 3: LR/SC erhöht einen Wert um zwei (6 Punkte)

1. Hardwaremodell: Zwei Harts lesen jeweils Zähler zehn, addieren lokal zwei und schreiben beide zurück. Notieren Sie eine Folge und den Endwert statt des erwarteten 14. (2 P)
2. Wie unterscheidet sich ein gewöhnlicher ungeschützter C-Zähler von diesem vorgegebenen Hardwareablauf? Genügen getrennte atomare Loads und Stores? (1 P)
3. LR/SC soll dieselbe Addition atomar ausführen. Beide LR lesen anfangs zehn. SC von Hart 1 mit zwölf gelingt, SC von Hart 0 mit zwölf scheitert. Hart 0 wiederholt LR, berechnet neu und sein nächster SC gelingt. Geben Sie gelesene Werte, geschriebene Werte, SC-Status und Endwert an. (2 P)
4. Warum darf Hart 0 nicht einfach den alten Wert zwölf erneut speichern? Beweist jeder SC-Fehler einen fremden Store? (1 P)

Ihre Lösung:

Aufgabe 4: Übergabe und ein dreiteiliger Wartekreis (6 Punkte)

1. Produzent schreibt einmalig gewöhnliche data 73 und danach atomar ready sieben mit Release. Konsument liest genau diese sieben mit Acquire und danach data; keine späteren Überschreibungen. Ergebnis und Synchronisationsvoraussetzung? (2 P)
2. Separat sind beide Objekte atomar und alle Zugriffe relaxed; data beginnt bei -1. Ist ready sieben mit data -1 erlaubt? Was wäre bei gewöhnlicher ungeschützter Payload anders? (1 P)
3. A hält X und wartet auf Y, B hält Y und wartet auf Z, C hält Z und wartet auf X. Zeichnen Sie den Kreis und nennen Sie eine globale Erwerbsreihenfolge. Darf ein Timeout fremde Sperren löschen? (2 P)
4. Ein Treiber hält P und wird unterbrochen; seine ISR wartet auf P. Warum genügt eine atomare Lockoperation nicht? Hält lokale Interruptsperre andere Harts fern? (1 P)

Ihre Lösung:

Aufgabe 5: Drei Kerne und ungleiche Teilpakete (4 Punkte)

Ein-Kern-Zeit 150 Einheiten, serieller Anteil 0,3, paralleler Anteil 0,7. Zunächst perfekte Verteilung ohne Zusatzaufwand. $S(N) = 1/(s + (1 - s)/N)$, Effizienz $E = S/N$.

1. Zeit, Speedup und Effizienz bei drei Kernen? (1 P)
2. Zeit, Speedup, Effizienz bei sechzig Kernen und theoretische Speedupgrenze? (1 P)
3. Bei drei Kernen kommen sechs Zeiteinheiten hinzu. Neue Zeit, Speedup und normierter Zusatz $h(3)$? (1 P)
4. Ohne diesen Zusatz: Die 105 parallelen Einheiten sind als 15, 25, 65 verteilt. Parallele Abschlusszeit, Gesamtzeit und Speedup? (1 P)

Ihre Lösung:

Aufgabe 6: Ergebnisse veröffentlichen und übernehmen (6 Punkte)

C11-Lehrmodell: Zwei Worker lesen dieselbe unveränderliche Eingabe `[3, 6, 9, 12]`. Worker 0 addiert die Elemente mit Index null und eins, Worker 1 die mit Index zwei und drei. Jeder schreibt genau einmal ausschließlich seinen eigenen Eintrag im gewöhnlichen `int32_t teil[2]`. Die atomaren Meldungen `atomic_uint fertig[2]` beginnen bei null. Ein separater Sammler liest die Ergebnisse erst, wenn beide Meldungen eins sind. Alle Objekte sind vor dem Start initialisiert; kein weiterer Synchronisationsweg und kein Wiederverwenden der Meldungen. Die Werte laufen nicht über.

Die C11-Ausschnitte gehören zu verschiedenen Ausführungsfäden:

```
/* Worker i, i ist 0 oder 1: */
teil[i] = lokale_summe;
atomic_store_explicit(&fertig[i], 1u, memory_order_relaxed);

/* Sammler, gesamt wird nur von ihm verwendet: */
if (atomic_load_explicit(&fertig[0], memory_order_acquire) == 1u &&
    atomic_load_explicit(&fertig[1], memory_order_acquire) == 1u)
    gesamt = teil[0] + teil[1];
```

1. Welche Teilsummen und welcher Gesamtwert sind beabsichtigt? (1 P)
2. Beurteilen und reparieren Sie die Veröffentlichung. Zeichnen Sie die nötige Ordnungsbeziehung von jedem Worker bis zum Lesen seines Ergebnisses. Darf dem fehlerhaften C11-Programm eine bestimmte Liste möglicher Gesamtwerte zugeschrieben werden? (2 P)
3. Separates Layoutmodell: 64-Byte-Cachezeilen, zwei Vier-Byte-Ergebnisse bei `0x20002000` und `0x20002004`; die Kontrollfelder liegen in anderen Zeilen. Welche zusätzliche Kopienkoordination kann trotz verschiedener Ergebnisobjekte entstehen? Wählen Sie eine passende nächste Zeilenadresse für das zweite Objekt. Ersetzt diese Trennung die Reparatur aus Teil 2? (2 P)
4. Worker 1 erreicht seine Meldung nie. Garantiert eine korrekt geordnete Veröffentlichung trotzdem die vollständige Ergebnisübernahme durch den Sammler? Welche zusätzliche Entwurfsentscheidung fehlt? (1 P)

Ihre Lösung:

Optional: am Rechner prüfen

Erweitern Sie `src/lab09_atomics.c` um drei Worker mit je 7000 atomaren Additionen: Der Zähler beginnt bei null und muss nach allen Joins 21000 enthalten. Prüfen Sie außerdem die einmalige Veröffentlichung von Payload 73. Führen Sie kein gewöhnliches ungeschütztes C-Race als Zahlenbeweis aus.